

Layered Agentic AI Workflows for Human-Centric Software Development

CrashOverride

Christopher Mills, christopher.mills@colostate.edu · Agentic AI Developer · Researcher

Amber Ferrell, amber.ferrell@colostate.edu · Developer · Researcher

Department of Computer Science, Colorado State University, Fort Collins, CO, USA

ABSTRACT

Agentic AI has become a major part of everyday software engineering, and human developers must remain at the center of control for these systems. It is unclear how using agentic systems affects software correctness and a human's ability to supervise them. We will build a human-centric coding assistant to compare multiple agentic workflows and a developer's understanding, confidence, workload, and decision-making process. The results will show whether additional agent layers meaningfully improve software quality and human oversight or introduce unnecessary complexity.

KEYWORDS

Agentic AI; Human-Centric; Multi-layer; AI workflow; Code Generation; AI workflow

1 INTRODUCTION AND MOTIVATION

The way software is developed has changed rapidly in the past few years, starting with AI chatbots that evaluated and generated small segments of code at a time. Today, we see more advanced AI use through agentic systems that can go through entire repositories, edit files, and collaborate to generate software solutions. It is very important that human developers stay in control by understanding, evaluating, and deciding which AI-generated changes to implement into their systems.

AI is a powerful tool that can generate solutions much faster than humans generally can. However, AI-generated solutions are not always correct, understandable, or trustworthy. With the introduction of agentic systems, we now see a wide range of specialized agents that can be assigned to roles and perform different tasks. Multiple of these specialized agents can even work together to create structured workflows. These agentic architectures introduce new trade-offs for human interaction.

A single agent may quickly generate a solution for a developer while keeping the interaction relatively simple, but it may offer limited insight for users to evaluate if a solution is correct or trustworthy. Multi-layered agentic systems offer users additional information before the final result reaches them, potentially improving understanding, trust, and decision-making. However, too many or unnecessary layers can make the system hard to understand, generate unnecessary information, or create a false sense of security that a solution is correct when agents agree.

It is unclear whether these different workflows can meaningfully improve both software and a human developer's

ability to supervise AI-generated changes. These problems lead us to our question. How do layered agentic workflow structures affect software correctness, and a human developer's ability to understand, evaluate, and trust AI-generated changes?

2 RELATED WORK

2.1 Multi-Agent Frameworks

Recent work has demonstrated how multi-agent structures perform using specialized LLM agents for software development. MetaGPT assigns specific roles and skills into Standard Operating Procedures (SOP's) to establish basic workflows [1]. Similarly, AgentCoder experimented with the effectiveness of adding specialized test design and test execution agents to generate test cases and provide feedback for refinement [2]. ChatDev further demonstrates this by assigning specialized agents to different stages of development and allowing them to collaborate through structured communication [3]. Together, these systems suggest that specialized roles and multi-layered architectures can structure tasks and produce better outputs, rather than relying on a single agent to do all of the work.

2.2 AI and Human-Centric Approaches

While multi-agent systems can improve task organization, added complexity can make their behavior harder for humans to understand. DiLLS addresses this by organizing agent behavior into layered summaries that help developers diagnose failures without relying on raw logs alone [4]. Dibia et al. compare automated code-generation metrics with human judgments and find that system-level performance does not always align with what developers consider valuable [5]. Together, these studies show that AI coding systems should be evaluated not only by technical performance, but also by how well humans can understand and assess their outputs.

While previous work has studied multi-agent software development and human interaction with AI systems, there has not been research directly comparing how adding individual agent layers affects both system performance and the human developer. Our project addresses this gap by comparing a programmer-only workflow, a programmer-reviewer workflow, and a planner-programmer-reviewer workflow while keeping the underlying model, task environment, and interface consistent. We evaluate whether these additional layers improve code correctness

and help developers better understand, trust, and evaluate AI-generated changes.

3 APPROACH

We will compare three structured agentic AI workflows where users will make feature changes to an existing baseline application. Developers will receive detailed feature requirements of varying difficulty, then use each agentic workflow to generate their own prompts as input. Once the agent(s) have generated a solution, the user will be able to interact and make decisions about what to do with the code.

The first workflow case will consist of a single programmer agent. The user will prompt the programmer agent to write code and then review the changes the agent made. Next, they will either accept, reject, or request additional changes. The user may request additional changes only once after reviewing the code. After that, the user must either accept or reject the final changes.

The second case uses both a programming agent and a reviewing agent. Similar to the first case, the user prompts the system, and the programmer generates code. The reviewing agent then reviews the changes and either provides detailed feedback to the programmer agent to improve the code based on its own requirements or passes it on for human review. After reviewing the new code and the review agent's feedback, the user can then accept, reject, or request additional changes.

Finally, the last case implements a planner agent before the programming agent begins writing code. In the final sequence, a human prompts the system, a planner divides tasks into a structured plan, the programmer generates code for each plan, the reviewer provides feedback, and the human user decides what to do with the changes.

Our experiment aims to combine elements of previous agentic systems through a layered, sequential workflow, specialized agents, and a focus on human interaction and experience. Unlike prior work, our main focus is on users' interaction with different suites of multi-layered agentic workflows. Users will be responsible for generating their own prompts based on feature requirements, analyzing agent feedback and code implementations, and making decisions about the code. This ultimately gives them full control over a human-centric agentic AI environment.

4 SYSTEM VISION AND TECHNOLOGY

We will code our system in Python using Anthropic's Claude Agents SDK and API. Each agent defined above will use the Claude model with different roles, instructions, and permissions. The mockup can be seen in *Figure 1* on the next page. The user will receive a small baseline application (e.g., a calculator app) and a 2D user interface describing a feature to implement, a prompt box, a structured display of agent feedback and output, a code diff, and accept/reject/request more buttons. The 2D user interface will also be coded in Python and use the Streamlit toolkit.

5 EVALUATION PLAN

The main independent variables in our experiment will be agent workflows discussed previously and feature difficulty. We will compare programmer, programmer + reviewer, and planner + programmer + reviewer workflows across easy, medium, and hard feature tasks. Our primary dependent variables will focus on the human interaction with the system, including confidence in the generated solution, trust that the solution is correct, understanding of the changes made, perceived workload, and whether participants accept or reject the generated code appropriately. We will also collect system-level measurements including hidden-test pass rate, regression failures, and agent execution time. We plan to recruit around 10 participants with programming experience and some familiarity with AI development tools. Each participant will complete nine total trials using copies of the same baseline repository. This study will be conducted only as a class project and will not require an IRB submission.

6 DELIVERABLES

By the end of November, we plan on having a project that includes three functioning agents using an underlying Claude model, a complete baseline repository built in Python for users to add features to, automated tests for implementation evaluation, and a 2D user interface that supports user-written prompts, agent feedback, code changes, and gathers user decisions, results, and feedback.

By the end of this class, we will complete the planned participant study and produce an analyzed dataset comparing correctness, human intervention, acceptance decisions, confidence, comprehension, workload, and execution cost between architectures.

7 CONCLUSION

This project examines whether AI agent workflows can improve both the overall correctness of AI-generated code and a developer's ability to supervise this code. By comparing the sequential addition of planning and review layers while keeping other system components consistent, the study will identify the benefits and trade-offs of adding more layers to agentic workflows. The class should expect to see a functioning multi-agent coding system with a human-centered interface, along with results from a participant study comparing code correctness, developer understanding, confidence, workload, and decision-making across the different architectures.

Figure 1: Caption. Say what the reader is looking at and what the interaction is, not just "system overview."

REFERENCES

- [1] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin

- Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In The Twelfth International Conference on Learning Representations.
- [2] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation. arXiv preprint arXiv:2312.13010.
- [3] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, 15174–15186.
- [1] Doug A. Bowman, Ernst Kruijff, Joseph J. LaViola Jr., and Ivan Poupyrev. 2004. 3D User Interfaces: Theory and Practice. Addison-Wesley, Boston, MA, USA.
- [2] Author One and Author Two. 2025. Title of the Paper in Sentence Case. In Proceedings of the CHI Conference on Human factors in Computing Systems (CHI '25). ACM, New York, NY, USA, Article 42, 14 pages.
- [4] Rui Sheng, Yukun Yang, Chuhan Shi, Yanna Lin, Zixin Chen, Huamin Qu, and Furui Cheng. 2026. DiLLS: Interactive Diagnosis of LLM-Based Multi-Agent Systems via Layered Summary of Agent Behaviors. In Proceedings of the CHI Conference on Human Factors in Computing Systems. ACM.
- [5] Victor Dibia, Adam Fourney, Gagan Bansal, Forough Poursabzi-Sangdeh, Han Liu, and Saleema Amershi. 2023. Aligning Offline Metrics and Human Judgments of Value for Code Generation Models. In Findings of the Association for Computational Linguistics: ACL 2023. Association for Computational Linguistics, 8516–8528.

Use of AI

Grammarly was used for grammar and style suggestions, and ChatGPT was used for feedback on writing, argument development, and help finding references.

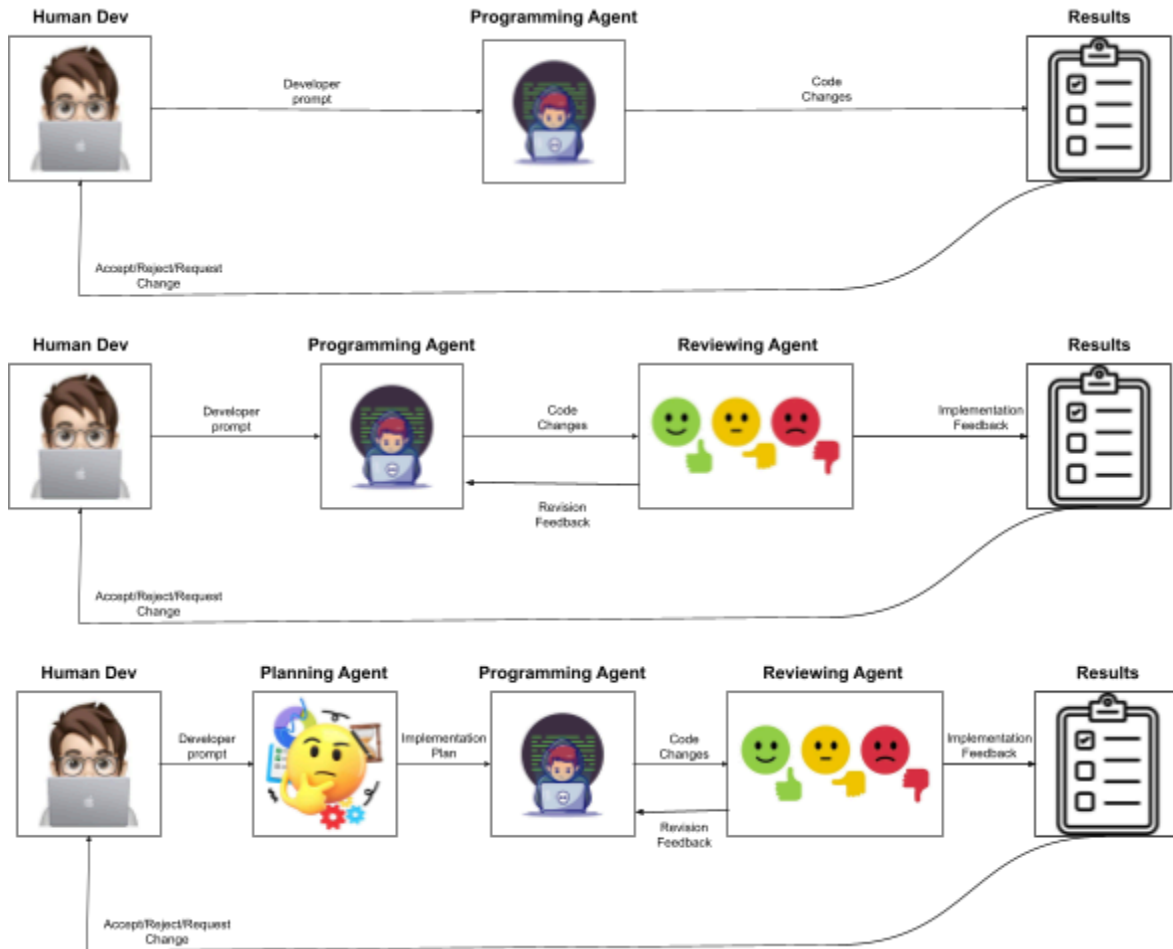


Figure 1: All three agentic workflow case diagrams, including planning, programming, and reviewing agents